

RA3D Software\SerialController.py

```

1  '''Serial controller. Sends and recevies serial commands'''
2  import serial
3  import serial.tools.list_ports
4  from serial import SerialException
5  import time, threading, queue, uuid
6
7  # TODO: Interesting bug that occurs but when connecting, disconnecting, then reconnecting,
the entire serial buffer gets messed up resulting in everything being read in as junk. Not
sure how it is happening nor how to fix.
8
9  class SerialController:
10
11      def __init__(self, root):
12          self.root = root
13
14          # General Connection Variables
15          self.port = ""
16          self.baud = 9600
17          self.board = None
18          self.boardConnected = False
19          self.running = False#To keep thread running only while conncted
20          # Threading Variables
21          self.serialThread = None
22          self.responseQueue = queue.Queue()
23          self.responseReady = False # Used as a flag to check if a response is ready to be
read
24          self.lastResponse = None # Stores whatever the last response was from serial until
requested
25
26          self.waiting_responses = set()
27          self.sendingSerial = False#Keep track to send serial so a response is not sorted
while sending
28          self.Errors = ["Es", "ER", "EL"] #Error codes
29          self.ErrorRaised = False #Flag to indicate if an error was raised, used to stop
waiting for responses if an error is raised
30          #Everything that will not be processed if immediately received
31          self.WaitToProcess = ["PO", "TL", "RE"] #currently these can only be 2 characters
32
33      def autoConnect(self):
34          self.root.statusPrint("Attempting automatic port connection")
35          self.root.portSelection.set("/dev/ttyACM0")
36          # Attempt the connection and if failed...
37          if self.serialConnect() == 0:
38              self.root.statusPrint("Failed to automatically connect to port /dev/ttyACM0.
Attempting /dev/ttyACM1")
39              self.root.portSelection.set("/dev/ttyACM0")
40              if self.serialConnect() == 0:
41                  self.root.statusPrint("Failed to automatically connect to port")
42                  self.root.portSelection.set("Select Port")

```

```

43
44     # Handles the "Connect/Disconnect" button being pressed to connect or disconnect the port
45     def serialConnect(self):
46         # Check if the serial controller is already connected to a board
47         if self.boardConnected == False:
48             # If not, connect to the port currently selected in the port dropdown
49             self.root.statusPrint("Attempting port connection")
50             self.connectPort(self.root.portSelection.get())
51             self.root.statusPrint(f"Connection Status: {self.boardConnected}")
52
53             # Perform a check to see if board was actually connected to
54             if self.boardConnected:
55                 self.root.connectButton.config(text="Disconnect") # Change button text
56                 self.root.portDropdown.config(state="disabled") # Disable port dropdown
57                 self.root.refreshCOMButton.config(state="disabled") # Disable refresh button
58                 self.root.portStatusLabel.config(text="Status: Connected") # Change port
status text
59                 #self.sendSerial("\n") #This is an attempt to fix that fact that the teensy
ignores the first command when just flashed
60                 return 1
61             else:
62                 return 0
63
64         else:
65
66             # If so, disconnect from the port
67             self.root.statusPrint("Disconnecting from port")
68             self.disconnectPort()
69             self.root.statusPrint(f"Connection Status: {self.boardConnected}")
70             self.root.connectButton.config(text="Connect") # Change button text
71             self.root.portDropdown.config(state="readonly") # Enable port dropdown
72             self.root.refreshCOMButton.config(state="normal") # Enable refresh button
73             self.root.portStatusLabel.config(text="Status: Disconnected") # Change port
status text
74             return 0
75
76
77     def connectPort(self, port):
78         self.port = port
79         try:
80             ### TODO: Edge case not caught by this try is if connecting to port,
81             # unplug & replug board on same port, disconnect,
82             # then connecting again doesn't throw error even though it
83             # doesn't actually connect to the port and essentially bricks
84             # the program from connecting to another port until restarted
85
86             tempBoard = serial.Serial(self.port, self.baud)
87             self.root.statusPrint(f"Port {port} opened successfully")
88             self.board = tempBoard
89             self.board.reset_output_buffer()
90             self.board.reset_input_buffer()

```

```
91
92     time.sleep(1) # allow Arduino auto-reset
93     self.boardConnected = True
94     self.responseQueue = queue.Queue()
95
96     self.serialThread = threading.Thread(target=self.serialReader, daemon=True,
name="Serial Read Thread")
97     self.sortThread = threading.Thread(target=self.processResponses, daemon=True,
name="Serial Sort Thread")
98     self.running = True #Keeps threads running, must be true before starting
serialthread
99     self.serialThread.start()
100    self.sortThread.start()
101
102    except SerialException:
103        self.root.statusPrint(f"Failed to open port: {port}")
104
105    def disconnectPort(self):
106        if self.boardConnected == True:
107            self.running = False #serial is no longer running
108            time.sleep(0.1) # let threads exit cleanly
109
110            self.sendSerial("CL")#teensy command to close serial
111            try:
112                self.board.reset_input_buffer()
113                self.board.reset_output_buffer()
114                self.board.close()
115            except:
116                pass
117            self.boardConnected = False
118
119    # Repeatedly checks the serial port for new responses
120    def serialReader(self):
121        while self.running:
122            # Check if a board is connected
123            #print("serial thread running")
124            if self.boardConnected:
125                try:
126                    # If there are any bytes of data waiting...
127                    if self.board.in_waiting > 0:
128                        # Read them in and store them in the response queue
129                        # errors ignore prevents decode crashes from partial UTF-8 bytes.D
130                        data = self.board.readline().decode('utf-8', errors='ignore').strip()
131                        #if data
132                        if data:
133                            self.responseQueue.put(data)
134                except:
135                    pass
136            else:
137                time.sleep(0.05)
138
```

```

139         time.sleep(.002) #wait to give serial time to process
140         self.peekQueue()#Should remove all queue elements that should process immediately
141
142
143     #used for debugging to display the response queue
144     def printQueue(self):
145         if not self.responseQueue.empty():
146             self.root.terminalPrint("Response Queue:"+ str(list(self.responseQueue.queue)))
147         else:
148             self.root.terminalPrint("Queue is empty")
149
150     #remove all items from queue
151     def clearQueue(self):
152         self.responseQueue.queue.clear()
153
154     #queue is cleaned of a specified item by rebuilding the queue
155     def cleanQueue(self, item):
156
157         new_items = []
158         while not self.responseQueue.empty():
159             val = self.responseQueue.get()
160             if val != item:
161                 new_items.append(val)
162
163         for val in new_items:
164             self.responseQueue.put(val)
165         print("response queue cleaned")
166
167     #Peeks the queue for errors and response that shouldn't be waited for
168     def peekQueue(self):
169         #This is a safe operation for iterating through a queue
170         with self.responseQueue.mutex:
171             #for each item in queue
172             for item in list(self.responseQueue.queue):
173                 #if an error
174                 if item[:2] in self.Errors:
175                     #remove item from queue
176                     self.sortResponse(item)
177                     self.responseQueue.queue.remove(item)
178                     self.RaiseError(item) #Raise error to tell processes to not wait for a
response
179                     return
180                 if item[:2] not in self.WaitToProcess:
181                     self.root.terminalPrint(item + " not in wait to process")
182                     self.responseQueue.queue.remove(item)
183                     self.sortResponse(item)
184
185     #Raise Error from arm to notify other proceses
186     def RaiseError(self, error):
187         #set flag of printer

```

```

188     self.root.printController.flag = error
189     self.root.terminalPrint(f"Error {error} occurred")
190     self.ErrorRaised = True
191     #Allow other threads to process error
192     while self.waiting_responses:
193         time.sleep(0.01)
194     self.ErrorRaised = False
195     self.root.terminalPrint("Error processed, resuming normal operation")
196     self.clearQueue() #Clear queue of any responses that may have come in during error
197
198     #Advances the response queue every .002 seconds
199     def processResponses(self):
200         #exit this thread if serial is not connected
201         while self.running:
202             try:
203                 while not self.responseQueue.empty():
204                     #Only wait on certain responses
205                     if self.responseQueue.queue[0][:2] in self.WaitToProcess:
206                         responseToWaitFor = self.responseQueue.queue[0]
207                         #this seems to a little janky but I want the waiting_responses to be
clear for a moment before it
208                         #sorts a response, and gives things a moment to send then wait for a
response
209                         accTime = 0 #accumulated time
210                         endTime = 5 #how long to wait for
211                         #Will stop waiting if the queue has moved on
212                         while self.responseQueue.queue[0] == responseToWaitFor and accTime <
endTime:
213                             time.sleep(.01)
214                             accTime += .01
215                             #See if wait was triggered by same response
216                             if self.responseQueue.queue[0] == responseToWaitFor:
217                                 #if not self.waiting_responses and not self.sendingSerial:
218                                     response = self.responseQueue.get()
219                                     self.sortResponse(response)
220                                 #This means the print will estop regardless if it is waiting for a
response
221                                 #print("processing responses")
222                             except IndexError:
223                                 pass
224                             time.sleep(.002)
225
226     #Function process response because correct response is not guaranteed for a command
227     #reponse must be passed back to some functions because the response is out of the queue
228     #This function mainly serves as a backup for processing responses and should NOT be used
for normal processing of a response
229     def sortResponse(self, sortResponse):
230         #shorthand for ease
231         PC = self.root.printController
232         AC = self.root.armController
233         R = self.root

```

```

234         flag = None
235
236         #self.root.terminalPrint(f"Received Response: {sortResponse}")
237         self.root.terminalPrint("Warning! Sorting response\nReponses handled by sorting
signfy a problem")
238         if sortResponse[:5]== "Estop":
239             PC.printing = False
240             flag = "Estop"
241             PC.pauseAll() #This will pause the program regardless if ignore flags is on
242             R.warningPrint("Estop pushed, stopping print")
243             #self.cleanQueue("Estop")
244
245         if sortResponse[:2]== "ER":
246             R.statusPrint(f"Kinematic Error: {sortResponse[2:]}")
247             flag = "Kinematic Error"
248             AC.awaitingMoveResponse = False
249             #Have to clean queue because the arm sends a thousand ERs for some reason
250             self.cleanQueue("ER")
251
252         elif sortResponse[:2] == "EL":
253             R.statusPrint(f"Error Axis Fault, Out of Reach: {sortResponse[2:]}")
254             flag = "Axis Fault"
255             AC.awaitingMoveResponse = False
256             self.cleanQueue("EL")
257
258         elif sortResponse[:11] == "Turn Hazard":
259             self.root.statusPrint(f"Encountered Hazard Move Stopped: {sortResponse[2:]}")
260             self.root.warningPrint(f"Turn Hazard Encountered. Stopping Print")
261             flag = "Turn Hazard"
262             AC.awaitingMoveResponse = False
263
264         elif sortResponse[:13] == "Limit Pressed":
265             self.root.statusPrint(f"Limits switch: {sortResponse[2:]}")
266             self.root.warningPrint(f"Limit switch pressed")
267
268         elif sortResponse[:9] == "Step Difs":
269             pass
270
271         #NOTE IF these responses are handled here that means they missed the timeout window
or something else is wrong
272         else:
273             R.terminalPrint(f"Warning Received Unexpected Response: {sortResponse}\n Maybe
timeout window missed")
274             if sortResponse[:2] == "TL":
275                 #Limit switch test
276                 R.terminalPrint(f"Received TL Response: {sortResponse}")
277                 AC.limitTestUpdate(response=sortResponse)
278             elif sortResponse[:2] == "RE":
279                 R.terminalPrint(f"Received TL Response: {sortResponse}")
280                 AC.limitTestUpdate(response=sortResponse)
281             elif sortResponse[:4] == "Done":

```

```

282         #Home position finished
283         #Command set output on/off finished
284         #send position to arm
285         pass
286     elif sortResponse[:6] == "WTDone":
287         R.statusPrint("Wait command finished")
288     elif sortResponse[:4] == "echo":
289         R.statusPrint(f"Echo: {sortResponse[4:]}")
290     elif sortResponse == "\n":
291         pass
292
293     elif sortResponse[:3] == "POS" and AC.calibrationInProgress:
294         self.root.statusPrint("Position received from arm during calibration")
295         AC.calibrateArmUpdate(response=sortResponse)
296         self.root.terminalPrint("Sorting response...")
297     elif sortResponse[:3] == "POS" and AC.awaitingMoveResponse:
298         self.root.statusPrint("Position received from arm after move command")
299         #AC.moveUpdate(response=sortResponse)
300         self.root.terminalPrint("Sorting response...")
301     elif sortResponse[:3] == "POS" and AC.awaitingPosResponse:
302         self.root.statusPrint("Position received from arm after position request")
303         AC.processPosition(sortResponse)
304         self.root.terminalPrint("Sorting response...")
305     elif sortResponse[:3] == "POS":
306         AC.processPosition(sortResponse)
307         self.root.terminalPrint("Sorting response...")
308     else:
309         self.root.terminalPrint("Sorting response...")
310         R.statusPrint(f"Received Unrecognized Response: {sortResponse}")
311
312     if flag is not None:
313         PC.flag = flag
314
315     #Get next response in queue
316     def getNextResponse(self):
317         try:
318             return self.responseQueue.get_nowait()
319         except queue.Empty:
320             return None
321
322     #There are 2 ways responses are processed, they are processed at an interval by
processResponse or waited for
323     #Then the response is sorted and
324     #wait for a specific response but timeout if not received, standard timeout is 2 seconds
325     def waitForResponse(self, prefix, timeout=2):
326         start = time.time()
327         #so response is not taken from processResponses
328         #The unique code ensures that no other response can cancel another
329         uniqueCode = prefix + str(uuid.uuid4())
330         #Avoid any possible errors where timeout is negative

```

```

331         if timeout <= 0:
332             print("timeout is negative or zero")
333             timeout = 2
334         self.waiting_responses.add(uniqueCode)
335         #While less than timeout and no error is raised
336         while time.time() - start < timeout and self.ErrorRaised==False:
337             try:
338                 response = self.responseQueue.queue[0]#peek queue
339                 if response.startswith(prefix):
340                     #self.root.terminalPrint("response processed after waiting")
341                     response = self.responseQueue.get(timeout=.1)
342                     #Discard unique code because no longer waiting
343                     self.waiting_responses.discard(uniqueCode)
344                     return response
345             except (queue.Empty, IndexError):
346                 time.sleep(0.01)
347             pass
348
349         #Signifies error stopped the waiting for response
350         if self.ErrorRaised:
351             #Discard unique code because no longer waiting
352             self.waiting_responses.discard(uniqueCode) #discard after checking error raised
353             return "Error"
354         #Discard unique code because no longer waiting
355         self.waiting_responses.discard(uniqueCode)
356         #Signifies timeout occurred
357         return None
358
359     # Sends a string over serial to the connected port
360     def sendSerial(self, command):
361         self.sendingSerial = True
362         # Check if a board is connected
363         if self.boardConnected == False:
364             self.root.statusPrint("Failed to send command: No board connected")
365             return
366         #self.board.reset_input_buffer()
367         # If we aren't awaiting for a serial response, then send the command
368
369         if self.root.DebugMode: #This is debug mode because thousands of commands will be
sent while printing
370             self.root.statusPrint(f"Sending Command: {command}")
371             self.board.write(command.encode())
372             if self.root.DebugMode:
373                 self.root.terminalPrint("Command sent")
374             # Reset the input buffer
375             #self.board.reset_input_buffer()
376             self.sendingSerial = False
377         # stop waiting for response if something timedout
378
379     def refreshCOMPorts(self):

```

```
380     self.root.portList = self.getCOMPorts() # Create options list from found COM ports
381     self.root.portSelection.set("Select Port")
382     self.root.portDropdown["values"] = self.root.portList
383     self.root.portDropdown["textvariable"] = self.root.portSelection
384
385     # Returns a list of all COM ports with a device connected
386     def getCOMPorts(self):
387         # Get a full list of all comm ports of device
388         ports = list(serial.tools.list_ports.comports())
389         returnList = []
390         # If there are no ports, report that
391         if not ports:
392             self.root.statusPrint("No serial ports w/ connected devices found")
393         # If not...
394         else:
395             # Inform user of how many ports were found
396             self.root.statusPrint(f"Found {len(ports)} connected device(s):")
397             # Iterate through all ports
398             for port in ports:
399                 # Print out information about the port
400                 self.root.terminalPrint(f"Name: {port.device}\nDescription:
{port.description}\nID: {port.hwid}\n\n")
401                 # Check if the port is the GPIO serial port
402                 if port.device == "/dev/ttyS0":
403                     # If so, continue to the next port
404                     self.root.terminalPrint("Not appending device to dropdown")
405                     continue
406                 # Append the found port to the device list
407                 returnList.append(port.device)
408     return returnList
```